playground.py × Terminal

```
import openai
import secret
openai.api_key=secret.api_key

# if the code word is present we clear the contents
if '//clear//' in open('playground.txt').read():
  open("playground.txt", "w").close()
else:

  #open the file
  file = open("playground.txt", "r")
  #read each line
  filelines = file.readlines()
  #put all the lines in a single variable
  all_lines = ""
  for line in filelines:
      all_lines += line
  #close the file
  file.close()
  print(all_lines)
```

29% (11:1)

playground.txt ×

```
write a tagline for an ice cream shop.
Give me 5 movie recommendations.


The best ice cream in town!

1. The Godfather
2. The Shawshank Redemption
3. Forrest Gump
4. The Dark Knight
5. The Silence of the Lambs
```

79% (3:1)

Guide ×

Collapse

Response

On the top left we have our Python file. On the bottom left we have the empty text file where the user will be able to simply write their prompts and generate and get their responses from.

For starters, we are going to use the same structure we have been using before to generate our responses. First, let's get our libraries.

```
import os
import openai
import secret
openai.api_key=secret.api_key
```

We need to read our file and just put the contents of it as our prompt.

```
#open the file
file = open("playground.txt", "r")
#read each line
filelines = file.readlines()
#put all the lines in a single variable
all_lines = ""
for line in filelines:
    all_lines += line
#close the file
file.close()
```

TRY IT!

```
Command was successfully executed.
```

It is customary to close the file when done with it. To test our code, we will try a couple of things.
On the top left in our Python file try running the following code.

```
print("hello world")
```

TRY IT!

Now instead of printing `hello world` we are going to print the `all_lines`, to try and see the content of the `playground.txt` file.

```
print(all_lines)
```

TRY IT!

```
Command was successfully executed.
```

Since our playground has no content in it yet we are going to get a blank response. We should have the Codio IDE message that our code successfully ran.

Now let's try adding the following text on our `.txt` file (bottom left), and try running our file again since now our playground will not be empty.

```
write a tagline for an ice cream shop.
```

TRY IT!

```
write a tagline for an ice cream shop.
```

It should print out `write a tagline for an ice cream shop.`. Now that we know we have access to the contents of our file let's assign it as the prompt instead of printing it.

```
prompts = all_lines
```

TRY IT!

We typically employ the following code to prompt users and capture the AI response:

```
response = openai.Completion.create(model="text-davinci-002",
                                    prompt=prompts,
                                    max_tokens=256,
                                    top_p=0.1)
txt_response=response['choices'][0]['text'].strip()
print(txt_response)
```

TRY IT!

```
write a tagline for an ice cream shop.

The best ice cream in town!
```

Now that we can get it to generate a response on the content of our text file we are going to have it write down the response in that same text file. For that we need to first open our file then simply add the response in to the text file instead of printing it.

```
#open our file, and append to it
f = open("playground.txt", "a")
#write the response we want to append
f.write((txt_response))
#close our file
f.close()
```

TRY IT!

```
write a tagline for an ice cream shop.

The best ice cream in town!
```

From that we realize it generates our response on the same line. Delete the text in the playground file. Have it go back to simply:

```
Write a tagline for an ice cream shop
```

Before we write our response we are going to make sure it adds to a new line so we will change our code to the following:

```
#write the response we want to append
f.write(("\n"))
f.write((txt_response))
```

TRY IT!

Feel free to try it with different prompts. For example:

```
Give me 5 movie recommendations.
```

TRY IT!

```
write a tagline for an ice cream shop.
Give me 5 movie recommendations.


The best ice cream in town!

1. The Godfather
2. The Shawshank Redemption
3. Forrest Gump
4. The Dark Knight
5. The Silence of the Lambs
```

Let's just say it's a pain to keep erasing. Let's add some code that will clear our text file for us. We will use the key word `//clear//` in our playground to know to clear it instead of generating anything. In other words, if in a our file we see the `//clear//` keyword our program instead of generating anything will give us a blank page. Copy and paste the following in our code above the code we have already written. For the `else` case, we are going to make the code we have already written the contents for it.

```
# if the code word is present we clear the contents
if '//clear//' in open('playground.txt').read():
  open("playground.txt", "w").close()
else:
```

Opening a file in "write" mode clears it. Again we have to close it after we are done with it. Inside the else statement we can indent the rest of the code we have previously written

TRY IT!

What is the purpose of opening a file in `write` mode in Python?

- To read the file
- To add new content to the end of the file
- To clear the content of the file
- None of the above

The purpose of opening a file in "write" mode in Python is to write data to the file. If the file does not exist, it will be created. If the file does exist, its contents will be overwritten.

Check It!

Next